

what is node js ----->>>

node js is not a language. Node js is async language

this is a Server run time Environment (run JavaScript on server environment)

Node js can connect with database

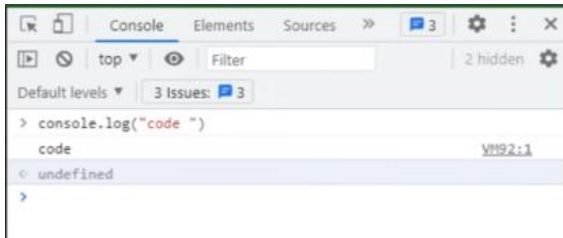
Code and Syntax is very similar to JavaScript, But not exact same

Node js free, open source

Node js use Chrome's V8 engine to execute code

Node js Version check ----->>> `node -v`

Npm js Version check ----->>> `npm -v`



With this example result shows undefined because with console.log only print the message not return anythings

```
> var a = 50
< undefined
> var b = 40
< undefined
> c = a+b
< 90
>
```

Here at the last equation result shows without undefined **because c is return the plus value of a and b**

Run node js from folder: ----->>>

Create a folder.

In side folder create index.js file.



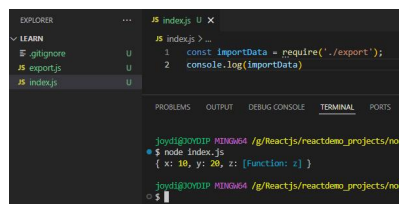
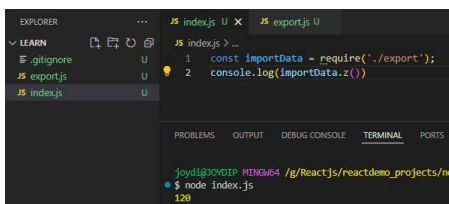
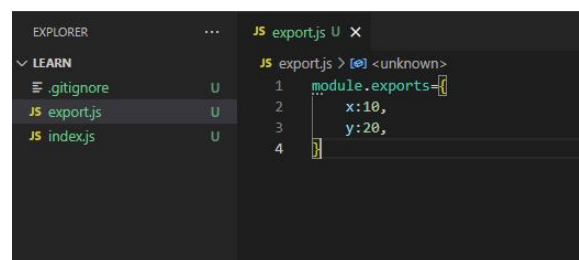
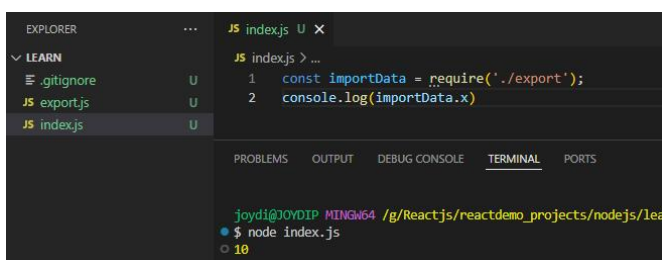
Difference between browser console and node js console ----->>>

Both console are different

browser console is a function of browser

node js console is a internal module of node js because node is not run in browser

Import file in node js



What is filter: -----

Filter is a function which helps to find a specific data from a array. It always works with array data only.

```
JS filter.js U X
JS filter.js > ...
1 const arr = [2,3,4,5,6,8,9,23];
2 arr.filter((item)=>{
3   console.log(item)
4 })
```

```
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects
$ node filter.js
2
3
4
5
6
8
9
23
```

```
JS filter.js U X
JS filter.js > ...
1 const arr = [2,3,4,5,6,8,9,23];
2 let filterResult = arr.filter((item)=>{
3   return item===5
4 })
5 console.log(filterResult);
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs
$ node filter.js
[ 5 ]
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs
$
```

```
JS filter.js U X
JS filter.js > filterResult > arr.filter() callback
1 const arr = [2,3,4,5,6,8,9,23];
2 let filterResult = arr.filter((item)=>{
3   return item > 5
4 })
5 console.log(filterResult);
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs
$ node filter.js
[ 6, 8, 9, 23 ]
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs
$
```

Types of module in Node JS -----

1. Core Module > 1.1> Global Module, 1.2> Non Global Module
2. External Module

In Node.js, **core modules** are built-in modules that come pre-packaged with the Node.js runtime. They provide essential functionality for various operations, such as working with files, streams, HTTP servers, cryptography, and more. These modules are directly available without requiring any additional installation or setup.

Characteristics of Core Modules

- ◆ **Pre-installed:** They are bundled with Node.js and don't require installation through npm.
- ◆ **Lightweight:** Optimized for performance and directly integrated with Node.js.
- ◆ **Always Available:** You can access them by simply using the `require()` function.

List of Node.js Core Modules: Other notable core modules include:

- ◆ **Buffer:** For working with binary data.
- ◆ **Stream:** For handling streaming data.
- ◆ **URL:** For URL resolution and parsing.
- ◆ **Timers:** For managing timeouts and intervals.
- ◆ **Zlib:** For compression and decompression.

Why Use Core Modules?

- ◆ No need to install additional dependencies.
- ◆ Highly efficient and optimized for Node.js.
- ◆ Reduce external dependencies in your project.

Global modules -----

In Node.js, global modules refer to Globally Installed Modules. These are modules installed system-wide and can be accessed from any project on your machine. They are not tied to a specific project directory.

To install a module globally, use the `-g` flag with npm:

Example: `npm install -g <module_name>`

Make a basic server and output on server: -----

In Node.js, the **http module** is a **core module** used to create HTTP servers and clients. It provides essential functionalities to interact with the **HTTP protocol**, enabling Node.js applications to handle web requests and responses.

Key Features of the http Module

- ◆ **Core Module:** Pre-installed with Node.js, no need to install it separately.
- ◆ **Web Server Creation:** You can create an HTTP server to handle client requests.
- ◆ **HTTP Client:** Allows making HTTP requests to external servers.
- ◆ **Customizable:** Offers flexibility to define how the server handles incoming requests and sends responses.

Basic Usage of the http Module

```
JS basic_server.js U X
JS basic_server.js > server.listen('4300') callback
1  const http = require('http');
2
3  const server = http.createServer((req, resp)=>{
4    resp.write("This is a simple created server");
5    resp.end();
6  });
7
8  server.listen('4300', ()=>{
9    console.log("server is running on port: 4300");
10 });
11
12
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/
$ node basic_server.js
server is running on port: 4300
```

```
← → ↺ ⓘ localhost:4300
⌵ | 🌐 Discord | Friends | 📧 Gmail | 📁 All Tools
```

This is a simple created server

what is package.json -----

In Node.js, **package.json** is a file that provides metadata about a project or module. It serves as the central configuration file for a Node.js project and includes information about the project, its dependencies, scripts, and other settings.

Creating a package.json File

create a package.json file using the following command: **npm init**

```
{
  "name": "learn",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}
```

Customize Package.json

```
{
  "name": "learn",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": ["nodejs", "example", "app"],
  "author": "author name",
  "license": "ISC"
}
```

```
"scripts": {
  "start": "node index.js",
  "test": "echo \"Error: no test specified\" && exit 1"
},
```

With this above start key we can run code by command: **npm start**

Create a Simple API: -----

```
const http = require("http");
http.createServer((req, resp)=>{
  resp.writeHead(200, { 'content-type' : 'application/json' });
  resp.write(JSON.stringify({ "name": "nodejs", "type": "javascript" }));
  resp.end();
}).listen(5000);
```

Pretty-print ☐

```
{"name":"nodejs","type":"javascript"}
```

Get Simple API Data from other file : -----

```
JS simple_api_datajs U X
JS simple_api_datajs > ...
1  const data = [
2    { "name": "nodejs", "type": "javascript" },
3    { "name": "reactjs", "type": "javascript" },
4    { "name": "angular", "type": "javascript" },
5  ];
6
7  module.exports = data;
```

```
JS simple_apijs U X
JS simple_apijs > http.createServer() callback
1  const http = require("http");
2  http.createServer((req, resp)=>{
3    resp.writeHead(200, { 'content-type' : 'application/json' });
4    resp.write(JSON.stringify(data));
5    resp.end();
6  }).listen(5000);
7
```

Pretty-print ☐

```
[{"name":"nodejs","type":"javascript"}, {"name":"reactjs","type":"javascript"}, {"name":"angular","type":"javascript"}]
```

Get input from command line : -----

```
JS command_line_get_inputjs U X
JS command_line_get_inputjs
1  // console.log(process);
2  console.log(process.argv);

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node command_line_get_input.js
[
  'C:\\Program Files\\nodejs\\node.exe',
  'G:\\Reactjs\\reactdemo_projects\\nodejs\\learn\\command_line_get_input.js'
]
```

```
JS command_line_get_inputjs U X
JS command_line_get_inputjs
1  // console.log(process);
2  console.log(process.argv);

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node command_line_get_input.js hello hi
[
  'C:\\Program Files\\nodejs\\node.exe',
  'G:\\Reactjs\\reactdemo_projects\\nodejs\\learn\\command_line_get_input.js',
  'hello',
  'hi'
]
```

```
JS command_line_get_inputjs U X
JS command_line_get_inputjs
1  // console.log(process);
2  console.log(process.argv[2]);

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/
$ node command_line_get_input.js hello hi
hello
```

Create a file with text using fs core module and process.argv -----

```
JS command_line_get_input.js M X
JS command_line_get_input.js > ...
6
7 // create file with process
8 const fs = require("fs");
9 const input = process.argv;
10 fs.writeFileSync(input[2], input[3] ); // input[2] => file name, input[3] => text
11
12
13 You, 12 seconds ago • Uncommitted changes

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node command_line_get_input.js test.txt 'simple text write in file using FS and process.argv'

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
○ $
```

```
JS command_line_get_input.js M test.txt U X
test.txt
1 simple text write in file using FS and process.argv
```

For Create a file -----

`fs.writeFileSync`

For remove a file -----

`fs.unlinkSync('filename')`

Add and remove file using fs module and process -----

```
JS command_line_get_input.js M ●
JS command_line_get_input.js > ...
5
6
7 // add and delete file with process
8 const fs = require("fs");
9 const input = process.argv;
10 if(input[2] === "add"){
11     fs.writeFileSync(input[3], input[4]);
12 }else if(input[2] === "delete"){
13     fs.unlinkSync(input[3]);
14 }else{
15     console.log("enter value")
16 }
17
18 You, 2 minutes ago • Uncommitted change

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node command_line_get_input.js add add.txt 'simple text write in file using FS and process.argv'

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node command_line_get_input.js delete add.txt

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
○ $
```


Get folder path: -----

```
JS folder_path.js U X
JS folder_path.js > [0] dirpath
1  const fs = require('fs');
2  const path = require('path');
3  const dirpath = path.join(__dirname)
4  console.log(dirpath);

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node folder_path.js
G:\Reactjs\reactdemo_projects\nodejs\learn
```

```
JS folder_path.js U X
JS folder_path.js > [0] dirpath
1  const fs = require('fs');
2  const path = require('path');
3  const dirpath = path.join(__dirname, 'innerfolder');
4  console.log(dirpath);

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node folder_path.js
G:\Reactjs\reactdemo_projects\nodejs\learn

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node folder_path.js
G:\Reactjs\reactdemo_projects\nodejs\learn\innerfolder
```

Create file in inner folder of project folder: -----

```
EXPLORER  ...  JS folder_path.js U X
└─ LEARN
  └─ innerfolder
     └─ text.txt
        node_modules
        .gitignore
        basic_server.js
        command_line_get_input.js
        export.js
        filter.js
        folder_path.js
        index.js
        package-lock.json
        package.json
        simple_api_data.js
        simple_api.js

JS folder_path.js > ...
1  const fs = require('fs');
2  const path = require('path');
3  const dirpath = path.join(__dirname, 'innerfolder');
4  const argv = process.argv;
5  //console.log(dirpath);
6
7  if(argv[2]=== 'add'){
8      fs.writeFileSync(`${dirpath}/${argv[3]}`, argv[4]);
9      console.log(`${argv[3]} file is created in "${dirpath}" folder`);
10 }else if(argv[2]=== 'remove'){
11     fs.unlinkSync(`${dirpath}/${argv[3]}`);
12     console.log(`${argv[3]} file is deleted from "${dirpath}" folder`);
13 }

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node folder_path.js add text.txt 'inner text'
text.txt file is created in "G:\Reactjs\reactdemo_projects\nodejs\learn\innerfolder" folder

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $
```

The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with a 'LEARN' folder containing 'innerfolder' and 'node_modules'. The code editor shows the content of 'folder_path.js', which uses the 'fs' and 'path' modules to create or delete a file in the 'innerfolder' directory based on command-line arguments. The terminal at the bottom shows the execution of the script with 'add' and 'remove' commands, resulting in file creation and deletion messages.

```
JS folder_path.js U X
JS folder_path.js > ...
1  const fs = require('fs');
2  const path = require('path');
3  const dirpath = path.join(__dirname, 'innerfolder');
4  const argv = process.argv;
5  //console.log(dirpath);
6
7  if(argv[2]=== 'add'){
8      fs.writeFileSync(`${dirpath}/${argv[3]}`, argv[4]);
9      console.log(`${argv[3]} file is created in "${dirpath}" folder`);
10 }else if(argv[2]=== 'remove'){
11     fs.unlinkSync(`${dirpath}/${argv[3]}`);
12     console.log(`${argv[3]} file is deleted from "${dirpath}" folder`);
13 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node folder_path.js add text.txt 'inner text'
text.txt file is created in "G:\Reactjs\reactdemo_projects\nodejs\learn\innerfolder" folder

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node folder_path.js remove text.txt
text.txt file is deleted from "G:\Reactjs\reactdemo_projects\nodejs\learn\innerfolder" folder

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
○ $
```

Create and get file list from a inner folder of project folder: -----

The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with a 'LEARN' folder containing 'innerfolder' and 'node_modules'. The code editor shows the content of 'read_dir.js', which uses the 'fs' and 'path' modules to create five text files in the 'innerfolder' directory and then reads the directory to log the file names. The terminal at the bottom shows the execution of the script, resulting in a list of file names being printed to the console.

```
JS read_dir.js U X
JS read_dir.js > fs.readdir() callback
1  const fs = require('fs');
2  const path = require('path');
3  const dirpath = path.join(__dirname, 'innerfolder');
4
5  for(i=0; i<5; i++){
6      fs.writeFileSync(`${dirpath}/textfile_${i}.txt`, 'text file content');
7  }
8
9  fs.readdir(dirpath, (err, files)=>{
10     console.log("show files list in console as array");
11     console.log(files);
12     console.log("get individual files name one by one");
13     files.forEach((file)=>{ console.log(file) });
14 });
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
• $ node read_dir.js
○ show files list in console as array
[
  'textfile_0.txt',
  'textfile_1.txt',
  'textfile_2.txt',
  'textfile_3.txt',
  'textfile_4.txt'
]
get individual files name one by one
textfile_0.txt
textfile_1.txt
textfile_2.txt
textfile_3.txt
textfile_4.txt

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$
```

Get file content / edit file content / rename file : -----

```
JS read_file.js U X
JS read_file.js > [?] filename
1  const fs = require('fs');
2  const path = require('path');
3  const dirpath = path.join(__dirname, 'innerfolder');
4  const filename = dirpath + '/textfile_0.txt'
5
6  fs.readFile(filename, (err, item) => {
7    if (!err) {
8      console.log(item);
9    }
10 })

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node read_file.js
<Buffer 74 65 78 74 20 66 69 6c 65 20 63 6f 6e 74 65 6e 74>

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$
```

```
JS read_file.js U X
JS read_file.js > ...
1  const fs = require('fs');
2  const path = require('path');
3  const dirpath = path.join(__dirname, 'innerfolder');
4  const filename = dirpath + '/textfile_0.txt'
5
6  fs.readFile(filename, 'utf8', (err, item) => {
7    if (!err) {
8      console.log(item);
9    }
10 })

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node read_file.js
<Buffer 74 65 78 74 20 66 69 6c 65 20 63 6f 6e 74 65 6e 74>

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node read_file.js
text file content

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$
```

```
JS read_file.js U X textfile_0.txt U
JS read_file.js > ...
1  G:\Reactjs\reactdemo_projects\nodejs\learn\read_file.js - Untracked
2  const path = require('path');
3  const dirpath = path.join(__dirname, 'innerfolder');
4  const filename = dirpath + '/textfile_0.txt'
5
6  fs.appendFile(filename, '00000 new text added 00001', (err) => {
7    if (!err) {
8      console.log("file content updated");
9    }
10 })
11

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node read_file.js
file content updated

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$
```

```
JS read_file.js U textfile_0.txt U X
innerfolder > textfile_0.txt
1  text file content 00000 new text added 00001

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node read_file.js
file content updated

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$
```

```
EXPLORER ... JS read_file.js U X
LEARN
innerfolder
  textfile_0.txt U
  textfile_1.txt U
  textfile_2.txt U
  textfile_3.txt U
  textfile_4.txt U
node_modules
.gitignore
JS basic_server.js
JS command_line_get... M
JS export.js
JS filter.js
JS folder_path.js U
JS index.js
package-lock.json
package.json
read_dir.js U
read_file.js U

JS read_file.js > ...
1  const fs = require('fs');
2  const path = require('path');
3  const dirpath = path.join(__dirname, 'innerfolder');
4  const filename = dirpath + '/textfile_0.txt'
5
6  // rename file name
7  fs.rename(filename, dirpath + '/textfile_rename.txt', (err) => {
8    if (!err) {
9      console.log("file name updated");
10 }
11 })
12
13

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$
```

```
EXPLORER ... JS read_file.js U X
LEARN
innerfolder
  textfile_1.txt U
  textfile_2.txt U
  textfile_3.txt U
  textfile_4.txt U
  textfile_rename.txt U
node_modules
.gitignore
JS basic_server.js
JS command_line_get... M
JS export.js
JS filter.js
JS folder_path.js U
JS index.js
package-lock.json
package.json
read_dir.js U
read_file.js U
simple_api_data.js

JS read_file.js > ...
1  const fs = require('fs');
2  const path = require('path');
3  const dirpath = path.join(__dirname, 'innerfolder');
4  const filename = dirpath + '/textfile_0.txt'
5
6  // rename file name
7  fs.rename(filename, dirpath + '/textfile_rename.txt', (err) => {
8    if (!err) {
9      console.log("file name updated");
10 }
11 })
12
13

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$ node read_file.js
file name updated

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (demo-projects)
$
```

Buffer: temporary file location for save data using file system to store data node require some memory

Delete file : -----

EXPLORER

JS read_file.js U X

LEARN

innerfolder

textfile_1.txt U

textfile_2.txt U

textfile_3.txt U

textfile_4.txt U

textfile_rename.txt U

node_modules

.gitignore

basic_server.js

command_line_get... M

export.js

filter.js

folder_path.js U

index.js

package-lock.json

JS read_file.js > ...

1 const fs = require('fs');

2 const path = require('path');

3 const dirpath = path.join(__dirname, 'innerfolder');

4 const filename = dirpath+'textfile_0.txt'

5

6 fs.unlinkSync(dirpath+'textfile_rename.txt')

7

8

9

10

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (de

\$

EXPLORER

JS read_file.js U X

LEARN

innerfolder

textfile_1.txt U

textfile_2.txt U

textfile_3.txt U

textfile_4.txt U

node_modules

.gitignore

basic_server.js

command_line_get... M

export.js

filter.js

folder_path.js U

index.js

package-lock.json

JS read_file.js > ...

1 const fs = require('fs');

2 const path = require('path');

3 const dirpath = path.join(__dirname, 'innerfolder');

4 const filename = dirpath+'textfile_0.txt'

5

6 fs.unlinkSync(dirpath+'textfile_rename.txt')

7

8

9

10

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (de

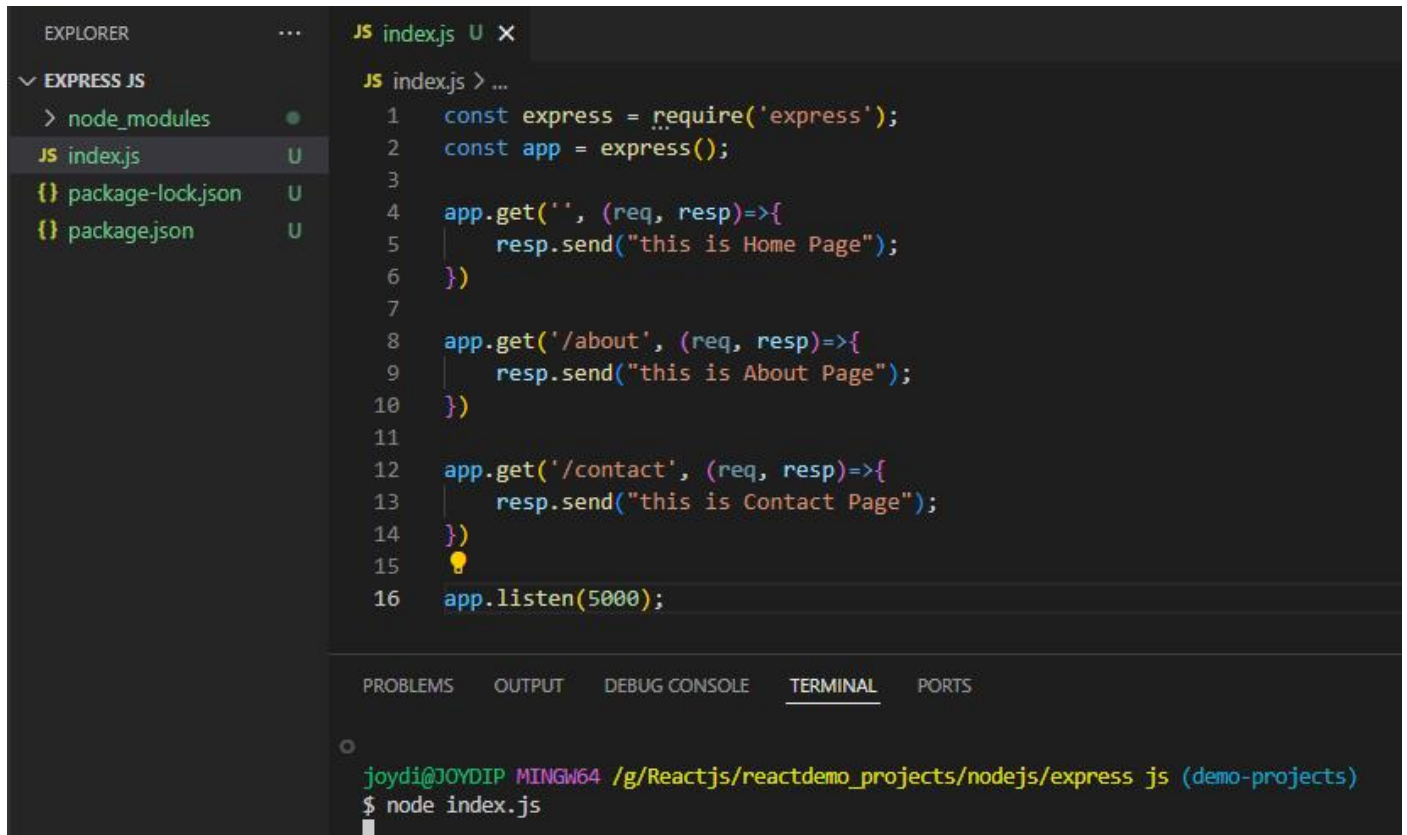
\$ node read_file.js

joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/learn (de

\$

----- Express JS -----

Create simple pages using Express -----

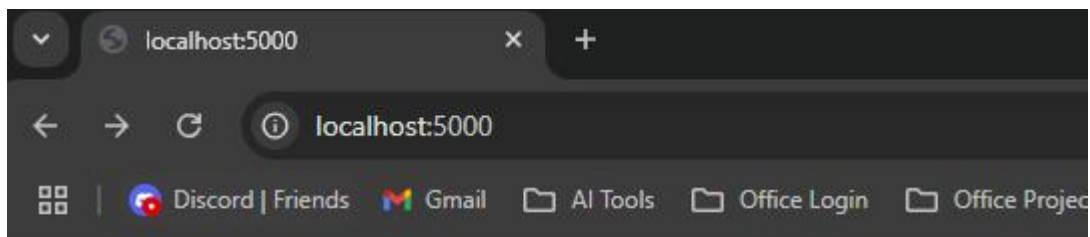


The screenshot shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'EXPRESS JS' with files 'node_modules', 'index.js', 'package-lock.json', and 'package.json'. The code editor shows the content of 'index.js' with the following code:

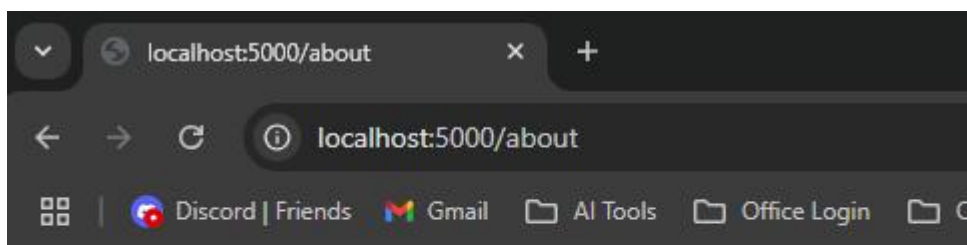
```
1 const express = require('express');
2 const app = express();
3
4 app.get('/', (req, resp)=>{
5   resp.send("this is Home Page");
6 })
7
8 app.get('/about', (req, resp)=>{
9   resp.send("this is About Page");
10 })
11
12 app.get('/contact', (req, resp)=>{
13   resp.send("this is Contact Page");
14 })
15
16 app.listen(5000);
```

The terminal at the bottom shows the command to run the application:

```
joydi@JOYDIP MINGW64 /g/Reactjs/reactdemo_projects/nodejs/express js (demo-projects)
$ node index.js
```



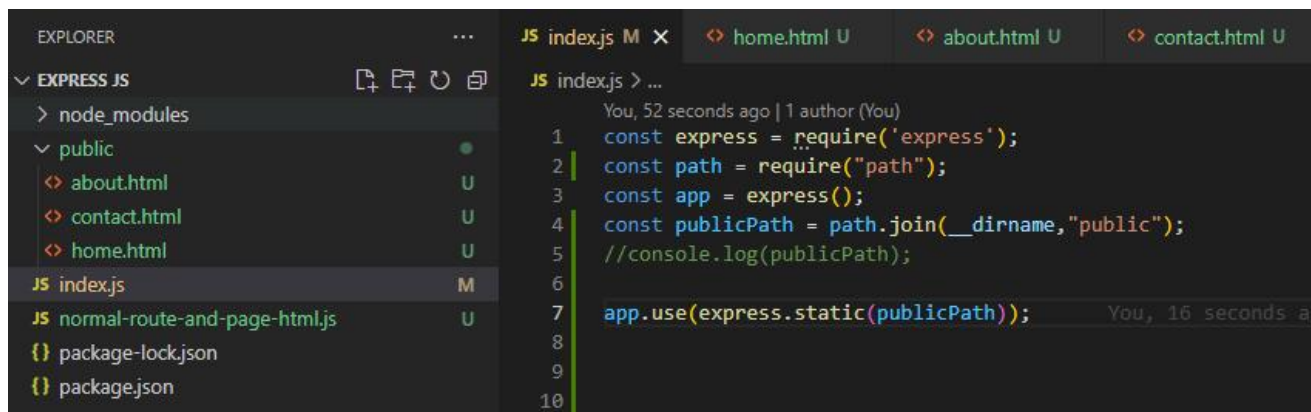
this is Home Page



this is About Page

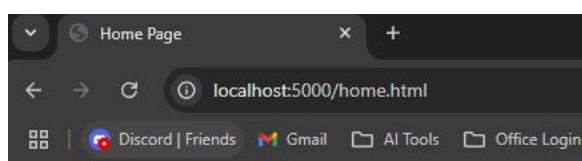
Load static HTML files from public folder -----

static method use for load static html pages from public folder

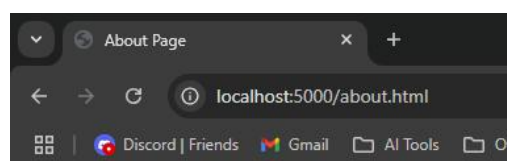


```
EXPRESS JS
├── node_modules
├── public
│   ├── about.html
│   ├── contact.html
│   └── home.html
├── JS index.js
├── JS normal-route-and-page-html.js
├── package-lock.json
└── package.json

JS index.js > ...
You, 52 seconds ago | 1 author (You)
1 const express = require('express');
2 const path = require("path");
3 const app = express();
4 const publicPath = path.join(__dirname, "public");
5 //console.log(publicPath);
6
7 app.use(express.static(publicPath));
8
9
10
```



Home Page



About Page

Remove extension from URL -----

```
const express = require('express');
const path = require("path");
const app = express();
const publicPath = path.join(__dirname, "public");

app.get('/', (req, resp) => {
  resp.sendFile(`${publicPath}/home.html`);
});

app.get('/aboutus', (req, resp) => {
  resp.sendFile(`${publicPath}/about.html`);
});

app.get('/contact-us', (req, resp) => {
  resp.sendFile(`${publicPath}/contact.html`);
});

app.get('*', (req, resp) => {
  resp.sendFile(`${publicPath}/404.html`);
});

app.listen(5000);
```

Template Engine -----

A **template engine** in web development is a tool that allows you to *generate dynamic HTML pages* by combining static templates with data.

Why Use a Template Engine?

- ◆ Generate HTML pages dynamically based on data
- ◆ Keep HTML templates separate from application logic, promoting clean code.
- ◆ Provides syntax for loops, conditionals, and variable interpolation directly within the HTML.

Popular Template Engines in Node.js

EJS (Embedded JavaScript)

Pug (formerly Jade)

Handlebars

Mustache

How to use ?

```
const express = require('express');
const path = require("path");
const app = express();
const publicPath = path.join(__dirname, "public");

// Set EJS as the template engine
app.set('view engine', 'ejs')

app.get('/', (req, resp) => {
  // resp.sendFile(`${publicPath}/home.html`);
  resp.render('home');
});

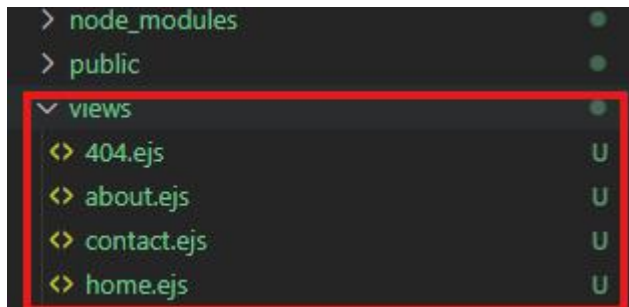
app.get('/aboutus', (req, resp) => {
  resp.render('about');
});

app.get('/contact-us', (req, resp) => {
  resp.render('contact');
});

app.get('*', (req, resp) => {
  resp.render('404');
});

app.listen(5000);
```

File Structure -----



Send data to ejs template page -----

```
const express = require('express');
const path = require("path");
const app = express();
const publicPath = path.join(__dirname, "public");

// Set EJS as the template engine
app.set('view engine', 'ejs')
app.get('/', (req, resp) => {
  const data = {
    name: 'Rahul Roy',
    email: 'rahul@gmail.com'
  }
  resp.render('home', {data});
});

app.get('*', (req, resp) => {
  resp.render('404');
});
```


home.ejs -----

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Home EJS Page</title>
</head>
<body>

  <h1>Home EJS Page</h1>
  <h2> Welcome, <%= data.name %> </h2>
</body>
</html>
```

Loop in ejs -----

```
app.get('/', (req,resp)=>{
  const data = {
    name: 'Rahul Roy',
    email: 'rahul@gmail.com',
    skills: ['html', 'css', 'javascript', 'node js']
  }
  resp.render('home', {data});
});
```

home.ejs -----

```
<ul>
  <% data.skills.forEach((item)=>{ %>
    <li><%= item %></li>
  <% }) %>
</ul>
```

Common header in ejs -----

```
<%- include("common/header") %>
<h1>Home EJS Page</h1>
<h2> Welcome, <%= data.name %> </h2>
<ul>
  <% data.skills.forEach((item)=>{ %>
    <li><%= item %></li>
  <% }) %>
</ul>
```

```
* For include HTML we need to use      <%- include("filename") %>

** For show JavaScript / data we need to use  <li><%= item %></li>
```

----- Middleware -----

In web development, **middleware** refers to functions that *sit between the request and the response* in a server's request-response cycle. Middleware functions are commonly used in frameworks like *Express.js to handle tasks such as logging, authentication, parsing, routing, and error handling.*

We can *access* and *modify request and the response* in *routes*

What Middleware Does

- ◆ **Intercepts Requests:** Middleware can inspect, modify, or terminate requests before they reach the final handler.
- ◆ **Processes Responses:** Middleware can modify or terminate responses before they are sent to the client.
- ◆ **Chains Execution:** Middleware functions can pass control to the next middleware in the stack using the `next()` function.

Middleware in Express.js

In Express.js, middleware functions are functions that have access to:

- ◆ **Request (`req`):** The incoming HTTP request object.
- ◆ **Response (`res`):** The outgoing HTTP response object.
- ◆ **Next (`next`):** A function to pass control to the next middleware.

Create Middleware -----

```
// create middleware
const routeMiddleware = (req, resp, next) => {
  console.log("middleware start working");
}

// use middleware
app.use(routeMiddleware);
```

Now if we run the `index.js` file we will see that page is continuously loading. So for that we need to add `next()` for goto further process like below and we will see in browser that page now loaded properly

```
// create middleware
const routeMiddleware = (req, resp, next) => {
  console.log("middleware start working");
  next();
}

// use middleware
app.use(routeMiddleware);
```

Middleware Example (Application Level Middleware) -----

```
// create middleware
const routeMiddleware = (req, resp, next) => {
  console.log("middleware start working");
  if(!req.query.age){
    resp.send("Please Provide Age")
  }else if(req.query.age < 18){
    resp.send("You are not eligible")
  }else{
    next();
  }
}
```

Middleware Example view -----

localhost:5000

Please Provide Age

localhost:5000/?age=17

You are not eligible

localhost:5000/?age=20

Common Header

Home EJS Page

Welcome, Rahul Roy

Types of Middleware -----

Type	Scope	Examples
Application-Level	Global	Logging, Authentication
Router-Level	Specific routes	/api middleware
Built-in	Provided by Express.js	express.json(), express.static()
Third-Party	External packages	morgan, helmet, cors
Error-Handling	Handles errors	Error logging
Request-Specific	Per route/method	Route-specific logic
Custom	Developer-defined	Authentication, Validation
Static	Serves static files	express.static()
Conditional	Executes based on conditions	API key check
Streaming	Processes file uploads	multer, busboy
Debugging/Monitoring	Debugging/Profiling	debug, express-status-monitor

Router Level Middleware Example -----

```
const express = require('express');
const app = express();
const path = require("path");
const publicPath = path.join(__dirname,"public");

// Set EJS as the template engine
app.set('view engine','ejs')

// create middleware
const routeMiddleware = (req, resp, next) => {
  console.log("middleware start working");
  if(!req.query.age){
    resp.send("<h2>Please Provide Age</h2>")
  }else if(req.query.age < 18){
    resp.send("<h2>You are not eligible</h2>")
  }else{
    next();
  }
}

app.get('', (req,resp)=>{
  const data = {
    name: 'Rahul Roy',
    email: 'rahul@gmail.com',
    skills: ['html', 'css', 'javascript', 'node js']
  }
  resp.render('home', {data});
});
```

```
app.get('/aboutus', routeMiddleware, (req,resp)=>{
  resp.render('about');
});

app.listen(5000);
```

With the above example we can not access **Aboutus page** without age validation

How to use Middleware from separate file -----

middleware.js

```
// create middleware
module.exports = routeMiddleware = (req, resp, next) => {
  console.log("middleware start working");

  if(!req.query.age){
    resp.send("<h2>Please Provide Age</h2>")
  }else if(req.query.age < 18){
    resp.send("<h2>You are not eligible</h2>")
  }else{
    next();
  }
}
```

index.js

```
const express = require('express');
const app = express();
const routeMiddleware = require('./middleware')
const path = require("path");
const publicPath = path.join(__dirname,"public");

// Set EJS as the template engine
app.set('view engine','ejs')

app.get('/aboutus', routeMiddleware, (req,resp)=>{
  resp.render('about');
});
```

Router Level Middleware type 2 example -----

middleware.js

```
// create middleware
module.exports = routeMiddleware = (req, resp, next) => {
  console.log("middleware start working");

  if(!req.query.age){
    resp.send("<h2>Please Provide Age</h2>")
  }else if(req.query.age < 18){
    resp.send("<h2>You are not eligible</h2>")
  }else{
    next();
  }
}
```

index.js

```
const express = require('express');
const app = express();
const routeMiddleware = require('./middleware')
const path = require("path");
const publicPath = path.join(__dirname,"public");
const route = express.Router();
```

```
// Set EJS as the template engine
app.set('view engine','ejs')

route.use(routeMiddleware);

route.get('/', (req,resp)=>{
  const data = {
    name: 'Rahul Roy',
    email: 'rahul@gmail.com',
    skills: ['html', 'css', 'javascript', 'node js']
  }
  resp.render('home', {data});
});

route.get('/aboutus', (req,resp)=>{
  resp.render('about');
});

app.get('/contact', (req,resp)=>{
  resp.render('contact');
});

app.use('/', route);

app.listen(5000);
```

With the above example we can not access **Aboutus page** and **Home page** without age validation

----- MongoDB -----

mongoDB version check: `mongod --version`

Add data in a collection using CLI:

```
db.collection01.insertOne({name:'ganesh', email:'ganesh@gmail.com'})
```

```
> db.collection01.insertOne({name:'ganesh', email:'ganesh@gmail.com'})
< {
  acknowledged: true,
  insertedId: ObjectId('67642e02124bf39bc0be8bd5')
}
```

The screenshot shows the MongoDB Compass interface. At the top, the breadcrumb navigation is 'localhost:27017 > nodejsDB > collection01'. Below this, there are tabs for 'Documents', 'Aggregations', 'Schema', 'Indexes', and 'Validation'. The 'Documents' tab is active, showing a single document with the following fields: `_id: ObjectId('67642e02124bf39bc0be8bd5')`, `name: "ganesh"`, and `email: "ganesh@gmail.com"`. The interface includes a query bar at the top with a 'Generate query' button, and a toolbar with buttons for 'ADD DATA', 'EXPORT DATA', 'UPDATE', and 'DELETE'.

The screenshot shows the 'Insert Document' dialog box in MongoDB Compass. The dialog is titled 'Insert Document' and has a close button (X) in the top right corner. It contains a text area with a placeholder that says 'Paste one or more documents here'. Below the text area, there is a 'VIEW' button and a 'Cancel' button. The 'Insert' button is highlighted in green. The dialog is open over the MongoDB Compass interface, which shows the same document as the previous screenshot.

The screenshot shows the MongoDB Compass interface after inserting three documents. The breadcrumb navigation is 'localhost:27017 > nodejsDB > collection01'. The 'Documents' tab is active, showing three documents. The first document is the same as the one in the previous screenshot. The second document has the following fields: `_id: ObjectId('67642edfecda8a0332dd5bd5')` and `name: "tetetet"`. The third document has the following fields: `_id: ObjectId('67642f28ecda8a0332dd5bd6')` and `name: "Rahul Bose"`. The interface includes a query bar at the top with a 'Generate query' button, and a toolbar with buttons for 'ADD DATA', 'EXPORT DATA', 'UPDATE', and 'DELETE'.

Check inserted data using CLI:

```
db.collection01.find()
```

Clear CLI: `ctrl + I`

-----Connect MongoDB with Node JS -----

```
const {MongoClient} = require('mongodb');
const mongourl = 'mongodb://localhost:27017';
const client= new MongoClient(mongourl);

async function getData ()
{
    // connect with mongodb
    let result = await client.connect();
    // connect with database
    let db = result.db('nodejsDB');
    // select collection
    let collection = db.collection('collection01');
    // handle promise
    let response = await collection.find({}).toArray();
    console.log(response);
}
getData();
```

```
function getData ()
{
    let result = client.connect();
}
```

(method) MongoClient.connect(): Promise<MongoClient>
Connect to MongoDB using a url
@remarks
Calling connect is optional since the first operation you perform will ca

`client.connect()` always response a promise. Promise use for handle the delay of data fetching

For resolve this we will use **async** and **await**

```
async function getData ()
{
    let result = await client.connect();
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
},
{ _id: new ObjectId('67642edfecda8a0332dd5bd5'), name: 'tetetet' },
{ _id: new ObjectId('67642f28ecda8a0332dd5bd6'), name: 'Rahul Bose' }
]
[nodemon] restarting due to changes...
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
[
  {
    _id: new ObjectId('67642e02124bf39bc0be8bd5'),
    name: 'ganesh',
    email: 'ganesh@gmail.com'
  },
  { _id: new ObjectId('67642edfecda8a0332dd5bd5'), name: 'tetetet' },
  { _id: new ObjectId('67642f28ecda8a0332dd5bd6'), name: 'Rahul Bose' }
]
```

Create config file for mongoDB connection -----

mongoDB.js

```
const {MongoClient} = require('mongodb');
const mongourl = 'mongodb://localhost:27017';
const client= new MongoClient(mongourl);

async function dbConnect ()
{
    // connect with mongodb
    let result = await client.connect();
    // connect with database
    let db = result.db('nodejsDB');
    // select collection
    return db.collection('collection01');
    // handle promise
}

module.exports = dbConnect
```

index.js

Handle Promise to fetch data from collection type 01 -----

```
// fetch data with promise type 01 -----
// dbConnect() return a promise. So we use then function
dbConnect().then((resp)=>{
    // toArray() also return a promise. So we use then function again
    //console.log(resp.find().toArray())
    resp.find().toArray().then((data)=>{
        console.log(data);
    });
});
})
```

Handle Promise to fetch data from collection type 02 -----

```
// fetch data with promise type 02 -----
const fetchData = async () => {
    //console.log("fetchData started");
    let data = await dbConnect();
    //console.log(data.find().toArray())
    data = await data.find().toArray(); // return a promise
    console.log(data)
}

fetchData()
```

-----Insert Data into MongoDB using Node JS -----

```
const dbConnect = require('./mongoDB');

const insert = async () => {
  console.log("insert function working");
  // db connection
  const db = await dbConnect();
  console.log(db);

  // insert one data
  // const insertData = await db.insertOne(
  //   { name: "Ashim", email: "ashim@gmail.com" }
  // );

  // insert many data
  const insertData = await db.insertMany([
    { name: "Sudip", email: "sudip@gmail.com" },
    { name: "Ashim", email: "ashim@gmail.com" }
  ]);

  // console.log(insertData);
  if(insertData.acknowledged){
    console.log("data inserted");
  }
}

insert();
```

-----Update Data into MongoDB using Node JS -----

```
const dbConnect = require('./mongoDB');

const update = async () => {
  console.log("update function working");
  // db connection
  const db = await dbConnect();
  console.log(db);
  let updateData = await db.updateOne(
    {name: "Jubin"},
    { $set:{ name: "Jubin 2" } }
  )
  if(updateData.acknowledged){
    console.log(updateData)
  }
}

update();
```